

Direct Action or Bounded Strategy Adaptation? Constrained Control Interfaces for LLM Agents in Partially Observable Multi-Agent Environments

Junrong Yi^{a,1}, Dian Jin^{a,1}, Haotian Chen^{b,1}

^a*School of Mathematical Sciences, Fudan University, Shanghai, China*

^b*School of Pharmaceutical Sciences, Fudan University, Shanghai, China*

Abstract

LLM agents are often asked to turn observations directly into executable actions. That design is convenient, but it gives the model control over every low-level decision variable and leaves little separation between strategic reasoning and operational constraints. We study an alternative in which the LLM updates a small persistent strategy state, while deterministic code converts that state into feasible actions. Both designs are evaluated in the same finite-horizon, partially observable multi-agent simulator. The comparison includes direct LLM control, direct control with additional history, bounded strategy adaptation, and bounded adaptation with additional history. In the bounded variants, the strategy state has six coordinates restricted to a fixed box, and all market actions pass through the same deterministic construction and risk checks. The testbed is a synthetic GPU-compute market with three heterogeneous providers, stochastic demand, customer reallocation, emergency transfer, backlog, and reputation feedback. The reported experiment uses one random seed, 12 rounds, and six scenarios. Across 864 LLM-controlled provider-round decisions, no fallback is recorded. In this run, the bounded variants obtain higher aggregate simulated profit than the direct variants, and added history is more helpful for bounded adaptation than for direct action generation. The result is evidence about this control interface in the present simulator, not evidence of statistical superiority or real-market performance.

Keywords: large language model agents, multi-agent systems, constrained decision making, partially observable environments, hierarchical control, agent-based simulation

¹These authors contributed equally to this work.

1. Introduction

LLM-based agents are now used in systems that repeatedly observe an environment, maintain some form of state, and choose actions. ReAct, Reflexion, Generative Agents, AgentBench, and SOTOPIA show that reasoning traces, memory, feedback, and environment-grounded evaluation can all affect interactive performance [14, 15, 16, 17, 18]. Most of this work asks how an agent should reason or what information it should remember. A separate design choice is more mechanical but just as consequential: how much of the executable action should be handed to the LLM in the first place?

Two simple controller designs illustrate the issue. A direct controller maps the current information state to quantities that the environment can execute. A layered controller asks the LLM to adjust a higher-level state and lets ordinary code translate that state into an action. Direct control is flexible, but the model must handle strategy, numerical choices, and feasibility at the same time. Layered control narrows the model’s role. The idea is related to hierarchical decision making, temporal abstraction, and constrained control, where high-level choices and low-level execution are separated [9, 10, 11].

For LLM agents, this control boundary has received less direct attention than prompting, memory, or planning. Recent work has begun to constrain action generation more explicitly. KnowAgent uses action knowledge to restrict planning trajectories, while action-decomposition methods reduce the difficulty of optimizing over large language-agent action spaces [12, 13]. These results motivate the question studied here: when the environment already has a structured action space, should the LLM choose the final action, or should it update a smaller strategic representation that is later converted into an action?

We examine this question in a synthetic market for high-end GPU compute. Three providers repeatedly choose prices and quantities under noisy demand, inventory limits, customer reallocation, backlog, reputation feedback, and inter-provider transfer. The market is deliberately synthetic. We use it because it gives a repeated, partially observable, multi-agent environment with clear operational constraints; it is not intended to reproduce the economics of any real cloud provider.

The simulator is held fixed and only the controller changes. We compare direct action generation, direct action generation with recent history, bounded strategy adaptation, and bounded strategy adaptation with recent history. In the bounded variants, the LLM updates six persistent strategy coordinates. Deterministic code

maps those coordinates to a forecast, price, and quantity, after which the same risk gate used by the other policies is applied. This interface has two limited guarantees that follow directly from the implementation: the strategy state stays inside its prescribed box, and the final action still obeys the simulator’s encoded feasibility rules.

The experiment is intentionally small: one LLM backend, one random seed, a 12-round horizon, three provider archetypes, and six scenarios. We therefore treat the results as a controlled comparison of controller behavior, not as a statistical benchmark or a model of real GPU markets. In the recorded run, the bounded variants earn more aggregate simulated profit than the direct variants. Extra history has little effect on direct control but a larger positive effect when it is used to update the bounded strategy state.

The main contributions are:

1. a common formulation of direct and bounded LLM controllers inside the same partially observable multi-agent process;
2. a bounded strategy interface that separates LLM-driven strategic updates from deterministic low-level action construction;
3. two implementation-level properties of that interface: bounded latent-state evolution and inherited low-level feasibility; and
4. a controlled comparison of context and control level, together with the experimental details and negative cases needed to interpret the reported results.

2. Related Work

2.1. Language-model agents

Language agents combine an LLM with observations, memory, tools, and an interaction loop. ReAct couples reasoning with actions [14]; Reflexion feeds verbal feedback from earlier attempts back into later ones [15]; and Generative Agents use stored experiences and reflection to support behavior over longer horizons [16]. AgentBench and SOTOPIA shift the emphasis from static language quality to what agents actually do inside an environment [17, 18]. Surveys of LLM-based autonomous and multi-agent systems identify memory, planning, tool use, coordination, and feedback as recurring components [20, 21].

Our concern is narrower. We are not proposing a new reasoning trace or memory mechanism. We study the interface between the LLM and the environment’s executable action space. KnowAgent is relevant because it limits planning through explicit action knowledge [12]; work on action decomposition is relevant because it treats a large action space as an optimization problem rather than leaving it entirely

to free-form generation [13]. We take the same general concern one step closer to execution by comparing direct action control with a persistent low-dimensional strategy state.

2.2. Hierarchical and constrained sequential decision making

Separating high-level intent from low-level execution is standard in sequential decision making. The options framework formalizes temporal abstraction in reinforcement learning [9]. Constrained Markov decision processes make restrictions part of the decision problem itself [10], and safe-RL work studies ways to improve performance without violating prescribed constraints [11].

The method studied here is not a new hierarchical-RL or safe-RL algorithm. We do not train a new policy with an RL objective. The connection is architectural: the LLM changes a slow strategy state, while deterministic code and explicit clipping, projection, and risk checks determine what can actually be executed. This also keeps the claims narrow. We show properties of the interface we implemented; we do not claim general safety, optimality, or convergence.

2.3. Multi-agent simulation

In multi-agent systems, an agent’s outcome depends on the joint behavior of other agents and on the environment dynamics [4]. Agent-based economic models use explicit rules for those interactions and have long been used to study heterogeneous behavior, market mechanisms, and policy interventions [1, 2, 3]. More recent examples include the AI Economist and EconAgent, which place learning-based or LLM-based agents inside economic simulations [5, 19].

We follow the same separation between agent policy and environment mechanics. Allocation, settlement, transfer, backlog, reputation, and feasibility are handled by shared simulator code. This matters for the comparison: a controller should not be credited for behavior that was inserted through a different settlement rule or allocation mechanism.

2.4. Cloud-resource markets as a testbed

Cloud computing has long been described as elastic, metered access to computational resources [6, 7]. Spot-instance markets provide a familiar case in which spare capacity is allocated through changing prices [8]. GPU services add scarce capacity, bursty training demand, service differentiation, and large demand fluctuations. Those features make the domain useful for a repeated decision experiment. Our simulator is nevertheless synthetic and is not calibrated to provider transaction data.

3. Problem Formulation

3.1. Finite-horizon partially observable multi-agent process

Consider N agents interacting over rounds $t = 0, \dots, T - 1$. Let the global market state be

$$s_t \in \mathcal{S}, \quad (1)$$

which contains latent demand, provider inventories, backlog, reputations, recent prices, transfer state, and other settlement variables. Agent i does not observe s_t directly. Instead it receives

$$o_{i,t} \sim \Omega_i(\cdot \mid s_t), \quad o_{i,t} \in \mathcal{O}_i, \quad (2)$$

where the observation mechanism includes noisy demand and visible market summaries.

Each provider chooses an action

$$a_{i,t} = (\hat{d}_{i,t}, p_{i,t}, q_{i,t}) \in \mathcal{A}_i(s_t), \quad (3)$$

consisting of a one-step demand forecast, quoted price, and offered quantity. The feasible set $\mathcal{A}_i(s_t)$ encodes provider-specific price bounds, capacity and inventory constraints, and discrete quantity rules. The joint action $a_t = (a_{1,t}, \dots, a_{N,t})$ induces state evolution

$$s_{t+1} \sim P(\cdot \mid s_t, a_t), \quad (4)$$

and reward $r_i(s_t, a_t, s_{t+1})$ for each provider. In the simulator the transition kernel is implemented by explicit demand generation, customer allocation, reallocation, transfer, backlog, settlement, and reputation-update routines.

For a policy π_i , the finite-horizon objective is

$$J_i(\pi_i, \pi_{-i}) = \mathbb{E} \left[\sum_{t=0}^{T-1} r_i(s_t, a_t, s_{t+1}) \right]. \quad (5)$$

We do not solve for an equilibrium. The objective of the experiment is comparative: evaluate alternative controller classes under the same transition and reward mechanisms.

3.2. Information histories

Let

$$h_{i,t} = (o_{i,0:t}, a_{i,0:t-1}, y_{i,0:t-1}) \quad (6)$$

denote the information history available to an agent, where $y_{i,t}$ summarizes realized settlement feedback such as profit, service, shortage, backlog, inventory, and competitor-relative price signals. The non-context policies receive the current observation and a limited regular observation window already present in the simulator. The context variants additionally receive a compressed six-round settlement history and derived risk signals.

4. LLM Control Interfaces

4.1. Direct action control

A direct LLM controller emits the executable decision variables:

$$\tilde{a}_{i,t} = F_{\theta}(x_{i,t}), \quad (7)$$

where $x_{i,t} = o_{i,t}$ for Direct LLM and $x_{i,t} = (o_{i,t}, \tilde{h}_{i,t})$ for Context, with $\tilde{h}_{i,t}$ denoting compressed history. The raw structured output is parsed and then passed through the common feasibility/risk layer R_i :

$$a_{i,t} = R_i(s_t, \tilde{a}_{i,t}). \quad (8)$$

A deterministic heuristic is available only as an API/parsing fallback.

4.2. Bounded strategy adaptation

The bounded controller introduces a persistent strategic state

$$z_{i,t} = (z_{i,t}^1, \dots, z_{i,t}^6) \in \mathcal{Z} = [0.05, 0.95]^6, \quad (9)$$

whose coordinates represent risk tolerance, price aggressiveness, demand sensitivity, inventory caution, shock responsiveness, and competitor reactivity.

After settlement, the LLM proposes a raw strategy update

$$\delta_{i,t} = F_{\theta}(x_{i,t}, z_{i,t}) \in [-0.1, 0.1]^6. \quad (10)$$

Role-specific weights, boundary damping, and coordinate-wise update caps produce an implemented increment $\Delta z_{i,t}$. The persistent state then evolves as

$$z_{i,t+1} = \text{Proj}_{\mathcal{Z}}(z_{i,t} + \Delta z_{i,t}). \quad (11)$$

The LLM does not emit the executable market action. Instead deterministic code constructs

$$\tilde{a}_{i,t} = g_i(o_{i,t}, z_{i,t}), \quad (12)$$

followed by the same shared risk layer

$$a_{i,t} = R_i(s_t, \tilde{a}_{i,t}). \quad (13)$$

Thus the LLM controls a low-dimensional strategic state rather than the full low-level action.

In the implementation, forecast and price modifications include terms of the form

$$\hat{d}_{i,t} = \hat{d}_{i,t}^H [1 + 0.18(z_{i,t}^3 - 0.5) + 0.03z_{i,t}^5 v_t], \quad (14)$$

$$p_{i,t} = p_{i,t}^H + 0.55(z_{i,t}^2 - 0.5) - 0.35z_{i,t}^6(p_{i,t}^H - \bar{p}_t) + 0.10(z_{i,t}^1 - z_{i,t}^4), \quad (15)$$

where $\hat{d}_{i,t}^H$ and $p_{i,t}^H$ are heuristic baseline quantities, v_t is observed market volatility, and $\bar{p}_t$ is average market price. Quantity is modified by risk, inventory caution, recent shortage, and inventory drag before rounding to the provider's legal quantity step.

4.3. Structural properties

The proposed architecture is intentionally simple enough that two useful properties follow directly from the implementation.

Proposition 1 (bounded strategy state). If $z_{i,0} \in \mathcal{Z}$ and each update follows Eq. (11), then $z_{i,t} \in \mathcal{Z}$ for every round t . Moreover, if every implemented coordinate increment is clipped to magnitude at most $m_k \leq 0.1$, then

$$\|z_{i,t+1} - z_{i,t}\|_\infty \leq 0.1. \quad (16)$$

Justification. Projection maps every proposed state back into $\mathcal{Z}$. Coordinate-wise clipping bounds each pre-projection increment, and projection onto the box cannot increase a coordinate displacement beyond the corresponding clipped proposal.

Proposition 2 (feasibility inheritance). Assume the deterministic action map followed by the risk layer satisfies

$$R_i(s, g_i(o, z)) \in \mathcal{A}_i(s) \quad (17)$$

for every legal s, o, z . Then every executable action produced by the bounded controller lies in $\mathcal{A}_i(s)$ regardless of the raw LLM update.

Justification. The LLM only changes z . Equation (13) composes the resulting state with g_i and R_i , whose codomain is the encoded feasible action set. The claim is therefore a property of the interface boundary rather than of the LLM itself.

These propositions do not establish optimality, stability of the full market dynamics, or robustness to adversarial model outputs. They formalize a narrower benefit: the LLM cannot directly bypass the encoded state box or the low-level action constraints.

5. GPU-Compute Market Testbed

5.1. Providers and customers

The environment contains three heterogeneous providers. Hyperscaler represents scale-oriented capacity, PremiumCloud emphasizes enterprise service quality, and SpotBroker emphasizes price-sensitive spot demand. Their initial parameters are shown in Table 1. These archetypes are hand-designed and should not be interpreted as calibrated representations of real firms.

Table 1: Provider initialization used in the simulator.

Provider	Role	Initial rep.	Base/floor price	Max new supply
Hyperscaler	Scale	0.65	4.60 / 4.60	120
PremiumCloud	SLA	0.85	5.40 / 4.40	120
SpotBroker	Spot	0.55	4.90 / 3.80	80

Customers are divided into training-burst demand (45%), enterprise-SLA demand (35%), and spot demand (20%). Provider choice uses a softmax attractiveness function combining price sensitivity, brand, reputation, delivery reputation, and segment-specific role preference. Segment price weights are 0.75/0.35/1.15, reputation weights are 0.80/1.45/0.45, and contract fractions are 0.10/0.75/0.05 for training, enterprise, and spot demand respectively.

5.2. Demand and observation model

Baseline latent demand follows

$$D_t^* = 180 + 0.6t + 18 \sin(2\pi t/7) + 10 \sin(2\pi t/30 + 0.3) + s_t + u_t, \quad (18)$$

$$u_t = 0.45u_{t-1} + \epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, 7^2), \quad (19)$$

followed by rounding and a lower bound of 50 units. Providers observe noisy demand rather than D_t^* directly; baseline observation noise has standard deviation 5.

5.3. Shortage recovery, payoff, and reputation

Unserved demand is first reallocated to providers with remaining capacity. Residual shortage may then trigger inter-provider emergency transfer. Contractual demand still unserved enters an SLA backlog queue, with old backlog served before new contractual demand in the next round.

Provider profit contains sales and backlog delivery, transfer income and expense, production cost, inventory holding and depreciation, SLA penalties, price-pressure cost, and repricing cost:

$$\pi_{i,t} = (s_{i,t} + b_{i,t}^{del})p_{i,t} + TR_{i,t}^{in} - TR_{i,t}^{out} \quad (20)$$

$$- C_{i,t}^{prod} - C_{i,t}^{inv} - C_{i,t}^{sla} - C_{i,t}^{pressure} - C_{i,t}^{menu}. \quad (21)$$

Reputation combines delivery, pricing, and cooperation components with weights 0.5, 0.3, and 0.2 and is updated by an exponential moving average with rate 0.25.

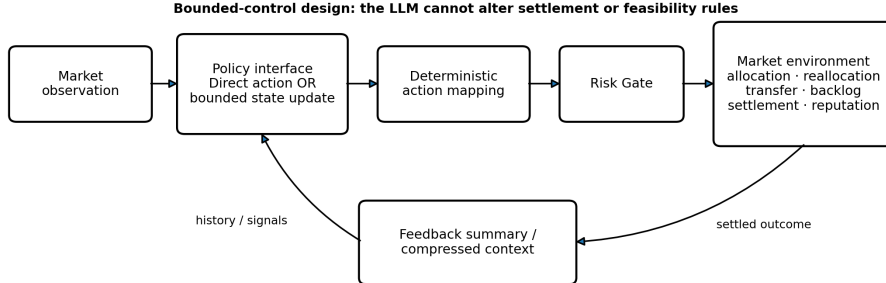


Figure 1: Control boundary in the simulator. The LLM influences either direct actions or a bounded strategic state, but settlement and feasibility are enforced by common deterministic code.

6. Experimental Protocol

6.1. Comparison design

We use the experiment to answer three concrete questions. **RQ1**: what changes when the LLM updates a bounded strategy state instead of choosing the executable action directly? **RQ2**: does recent settlement history help the two controller types

in the same way? **RQ3:** how do the controllers behave when market conditions or simulator support mechanisms are changed?

The four LLM policies can be read as a 2×2 comparison: context is either absent or present, and control is either direct or bounded. The heuristic policy serves as a non-LLM reference. Since every reported comparison uses the same single random seed, we use this structure only to organize the comparisons; it is not treated as a factorial statistical experiment.

6.2. Common experimental configuration

Table 2 collects the settings that stay fixed throughout the reported runs. Each simulation lasts 12 rounds and contains three providers and three customer segments. All four LLM controllers call the same `gemini-3-flash` backend with temperature 0, a 512-token output cap, one retry, and a 30-second timeout. Provider initialization, customer choice, demand generation, settlement, shortage recovery, reputation updating, and the final risk gate are otherwise identical across policies unless the scenario definition says otherwise.

6.3. AI-assisted implementation

Some portions of the research code were developed or revised with assistance from ChatGPT and Codex. The authors reviewed the AI-assisted code and verified the reported scientific claims and numerical outputs against the recorded experiment artifacts before using them in this manuscript. The experimental decision-making model itself is `gemini-3-flash`, whose role and configuration are described above.

6.4. Policy inputs

Table 3 shows exactly what differs at the policy input. Every LLM controller sees the current noisy demand signal, the provider’s own inventory and recent profit state, market-price and volatility information, relevant competitor and reputation signals, and the legal action ranges enforced by the harness. Context adds a compressed record of recent settlements and derived risk signals. Adaptive controllers also receive the current six-dimensional strategy state because that is the quantity they update. The market mechanism itself is unchanged.

6.5. Policy outputs and execution

The main difference is the output space. Direct controllers return the forecast, price, and quantity used for the current round, plus a textual rationale that is logged but not executed. After parsing, these values pass through the common risk and

Table 2: Fixed experimental setting for the reported study. Values are taken from the implemented simulator and the recorded experiment configuration.

Component	Setting
Randomness	Seed 7; stochastic latent demand and observation noise
Horizon	12 rounds; executed round indices 0–11
Providers	3: Hyperscaler, PremiumCloud, SpotBroker
Customer mix	Training 45%, enterprise-SLA 35%, spot 20%
Baseline demand	$180 + 0.6t + 18 \sin(2\pi t/7) + 10 \sin(2\pi t/30 + 0.3) + u_t$
Demand noise	$u_t = 0.45u_{t-1} + \epsilon_t, \epsilon_t \sim \mathcal{N}(0, 7^2)$
Observation noise	Gaussian, baseline s.d. 5
LLM backend	<code>gemini-3-flash</code>
LLM decoding	temperature 0; max output 512 tokens; 1 retry; 30-s timeout
Context memory	Up to 6 compressed post-settlement summaries for context variants
Bounded state	6 dimensions in $[0.05, 0.95]^6$; implemented per-coordinate update cap no larger than 0.1
Evaluation scope	5 policies $\times$ 6 scenarios; 36 provider-round records per policy-scenario run

feasibility checks. Bounded controllers return no executable price or quantity. They propose changes to six strategy coordinates; the update is damped, clipped, and projected, and ordinary code then constructs the market action from the resulting state.

6.6. Round-level execution order

A round is executed in a fixed order. The simulator first generates latent demand and the noisy observations available to providers. It then constructs the policy-specific input and obtains either a direct action proposal or a strategy update. Parsing, deterministic action construction where needed, and the shared risk gate are applied before customer demand is allocated. Shortages are reallocated, emergency transfer is used when the scenario permits it, and any remaining contractual shortage is carried into backlog. Payoffs and reputation are then updated and the round is logged. For adaptive policies, the persistent strategy state is updated only after settlement, so the current decision cannot use information from its own outcome.

Table 3: Information boundary across the four LLM controllers. A checkmark means the information is exposed to that controller in the implemented decision path.

Input / state information	Direct	Context	Adaptive	Context+Adaptive
Current noisy demand observation	✓	✓	✓	✓
Own inventory / recent profit state	✓	✓	✓	✓
Market-average price / volatility signal	✓	✓	✓	✓
Peer reputation / competitor information	✓	✓	✓	✓
Legal ranges and risk/fallback information	✓	✓	✓	✓
Compressed recent settlement history	—	✓	—	✓
Current six-dimensional strategy state	—	—	✓	✓

Table 4: Controller output and execution path.

Policy family	LLM output	Executed action path
Direct / Context	Forecast, price, quantity, rationale	Parse structured output → common risk/feasibility gate → market action
Adaptive / Context+Adaptive	Six-dimensional strategic update	Clip/damp/project state update → deterministic action construction → common risk/feasibility gate
Heuristic	None	Deterministic rule-based forecast, price, and quantity

6.7. Scenarios

We evaluate six scenarios, summarized in Table 5. The first four perturb market conditions; the final two are mechanism ablations. All unlisted parameters remain at baseline values.

6.8. Recorded runs and metrics

Each policy-scenario pair contributes $12 \times 3 = 36$ provider-round records. With five policies and six scenarios, the full experiment contains 1,080 simulator decisions, of which 864 are LLM-controlled. The recorded runs contain no direct-action fallback and no adaptive-state fallback.

We report simulated profit, fulfillment, final backlog, transfer volume, default flags, and one-step forecast mean absolute error (MAE). Profit is measured in the simulator’s internal payoff units and should not be read as money. The service

Table 5: Scenario interventions. The historical project label “supply shock” is retained for traceability, although the implementation creates supply pressure mainly through a positive demand shock and reduced reallocation efficiency.

Scenario	Intervention relative to baseline
Baseline	No intervention; all encoded market mechanisms active.
Price war	Higher price sensitivity; spot fraction 0.40; transfer markup 0.02; price-pressure cost 0.55.
Supply shock	From round index 8, demand shock +60; innovation s.d. 9; reallocation fill rate 0.60; maximum transfer 35.
High volatility	Innovation s.d. 16; observation-noise s.d. 10; stronger 7-round seasonality. The configuration also contains a +35 shock at round index 12, which lies outside executed indices 0–11 and therefore does not affect the reported run.
No reputation	Reputation contributions in customer choice and reputation updates are removed; other provider heterogeneity remains.
No transfer	Inter-provider transfer is disabled; customer reallocation remains active.

measures are reported alongside profit because the controller that earns the most need not provide the best service. MAE is included to separate forecasting quality from decision quality. With only one aligned seed, we do not report confidence intervals, p -values, or significance claims.

7. Results

7.1. Aggregate comparison

Table 6 sums the six scenarios for each policy. The two bounded controllers have higher aggregate simulated profit than the direct controllers in this recorded run. Context+Adaptive reaches 22,681, versus 19,770 for Direct LLM and 19,532 for the heuristic. It also has the lowest mean final backlog among the LLM policies.

The heuristic remains stronger on several service aggregates. This matters: the experiment does not show that LLM control dominates deterministic control on every objective. Rather, it shows a trade-off between profit-seeking behavior and service reliability within the encoded payoff and constraint system.

7.2. Context effect under direct and bounded control

The four LLM policies permit a descriptive 2×2 contrast. Let $J_{D,0}$ denote Direct LLM, $J_{D,C}$ Context, $J_{B,0}$ Adaptive, and $J_{B,C}$ Context+Adaptive, where

Table 6: Aggregate results across six scenarios in the reported single-seed experiment.

Policy	Profit	Fill	Defaults	Backlog
Heuristic	19,532	99.05%	6	0.69
Direct LLM	19,770	98.17%	14	2.23
Context	19,855	97.54%	27	2.24
Adaptive	22,349	98.35%	25	3.17
Context+Adaptive	22,681	98.46%	25	1.34

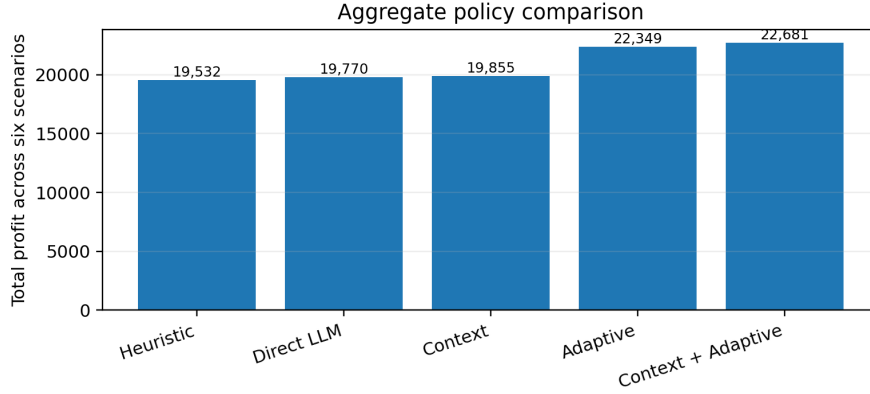


Figure 2: Aggregate simulated profit across six scenarios.

D/B indicate direct versus bounded control and $0/C$ indicate absence or presence of extra context.

Using aggregate profit from Table 6, the context effect under direct control is

$$\Delta_C^D = J_{D,C} - J_{D,0} = 19,855 - 19,770 = 85, \quad (22)$$

while the context effect under bounded control is

$$\Delta_C^B = J_{B,C} - J_{B,0} = 22,681 - 22,349 = 332. \quad (23)$$

The corresponding difference-in-differences interaction is

$$\Delta_{int} = \Delta_C^B - \Delta_C^D = 247. \quad (24)$$

This is not a statistical interaction test; it is a descriptive decomposition of one fixed-seed experiment. It nevertheless summarizes the qualitative pattern: additional context is associated with a larger profit change when routed through bounded strategic adaptation than when fed directly into low-level action generation.

7.3. Cross-scenario behavior

Figure 3 shows profit by policy and scenario. Context+Adaptive has the highest reported profit in five of six scenarios, with price war as the exception. This pattern should not be interpreted as robust superiority because the scenarios share one seed and one LLM backend.

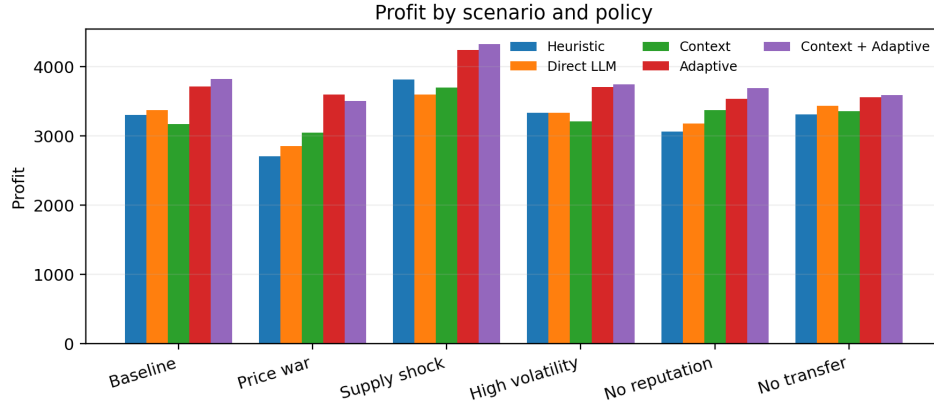


Figure 3: Simulated profit by scenario and policy.

7.4. Stress behavior under the project “supply shock”

The demand-driven supply-pressure scenario produces the clearest separation in backlog. Context+Adaptive reports profit 4326.6, fulfillment 96.08%, and final backlog 3.65, while the other LLM policies finish with backlog near 11. The adaptive variants also use more inter-provider transfer.

The higher profit does not imply that real supply disruptions benefit providers. The synthetic scenario raises demand and omits several real emergency procurement and capacity-expansion costs. The result only characterizes policy behavior under the encoded payoff.

7.5. Interpretable latent-state dynamics

The bounded interface exposes a persistent strategic state that can be inspected. Figure 5 shows the final Context+Adaptive state under the stress scenario. The three providers remain differentiated: Hyperscaler ends with high risk tolerance and demand sensitivity, PremiumCloud remains comparatively inventory-cautious, and SpotBroker becomes strongly shock- and competitor-reactive.

Supply-shock stress test (H=Heuristic, D=Direct, C=Context, A=Adaptive)

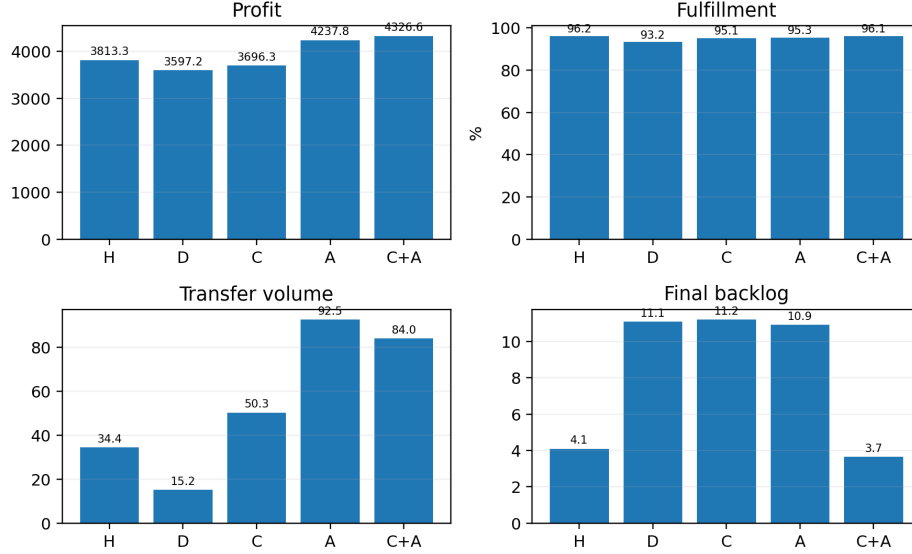


Figure 4: Stress-test outcomes under the project “supply shock” scenario.

This is evidence of role-preserving adaptation, not spontaneous discovery of market structure, because the providers start with different capacities, prices, brands, and strategy priors.

8. Analysis and Discussion

8.1. What changes when the LLM controls a strategy state

The difference between the two controller families is not the language model. It is the number and type of decisions exposed to it. Direct control asks the model to infer demand, interpret the competitive state, choose a price, choose a quantity, and stay within operational limits in one response. Bounded control asks it to move six persistent strategy coordinates; a deterministic map handles the numerical action that follows. The latter therefore removes some low-level degrees of freedom from the model and makes strategic changes persist across rounds.

That architecture is consistent with the pattern in the recorded results. The bounded variants earn more aggregate simulated profit, and adding settlement history helps more when the history is used to revise the persistent strategy state.

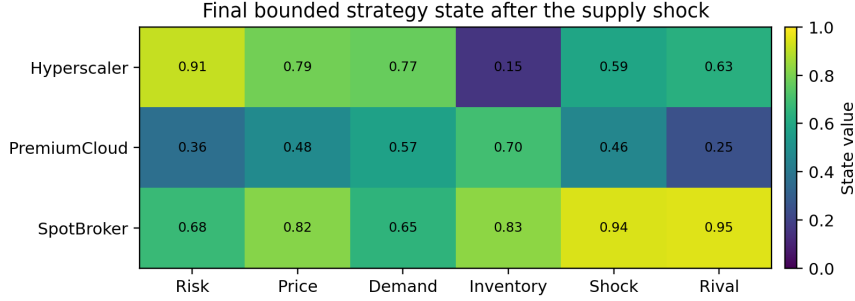


Figure 5: Final bounded strategy state after the stress scenario. Values are constrained to $[0.05, 0.95]$.

This is only a descriptive result from one seed. It does not show that a six-dimensional bottleneck is generally optimal, nor that the same pattern would hold with another model, horizon, or environment.

8.2. Where the comparison is less favorable

The results are not uniformly positive for the bounded design. In the price-war scenario, average prices move only modestly because price floors, role priors, and the risk gate limit sustained undercutting, and Context+Adaptive is not the best-performing policy. The 12-round high-volatility run is also too short to say much about rare or persistent demand excursions. The stress scenario leaves out emergency procurement, financing, long-term capacity expansion, and realistic GPU lead times. Removing reputation does not remove the other sources of provider heterogeneity, so that ablation should not be read as a homogeneous-agent experiment.

These cases show how much of the outcome still comes from the simulator design. A label such as “price war” does not guarantee a severe price war if other rules prevent it, and an advantage under the current payoff function may disappear under a different service objective or a longer horizon.

8.3. Limitations and validity threats

The clearest limitation is the single random seed. Temperature-zero decoding removes sampling variation from the LLM call, but demand and observation noise remain stochastic. Multi-seed runs are needed before making claims about variance, confidence intervals, significance, or stable policy rankings.

The horizon is also short. Twelve rounds exercise the feedback loop, but they do not establish long-run behavior of reputation, inventory, backlog, or the latent strategy state. The study uses one LLM backend and one synthetic market, so

model-specific and domain-specific effects cannot be separated. Provider roles and customer preferences are hand-designed rather than calibrated to transaction data.

Finally, the formal statements in Section 4 are deliberately limited. They describe what the implementation guarantees at the interface: bounded strategy coordinates and feasible actions after the deterministic gate. They do not imply reward optimality, equilibrium convergence, robustness to misspecified environment rules, or safety in a broader real-world sense.

9. Conclusion

This study compares two ways of placing an LLM inside a repeated decision loop. In the direct design, the model chooses executable market variables. In the bounded design, it updates a small persistent strategy state and deterministic code produces the final action. Both are evaluated under the same partially observable multi-agent simulator.

The bounded design makes the control boundary explicit. Its strategy coordinates are kept inside a fixed box, and the final action is still checked by the simulator’s deterministic feasibility layer. In the single-seed, 12-round experiment reported here, the bounded controllers earn more aggregate simulated profit than the direct controllers, and recent settlement history is more useful when it informs the strategy state than when it is attached directly to low-level action generation.

The evidence is intentionally limited. It comes from one model, one seed, one synthetic domain, and a short horizon. The main conclusion is therefore architectural rather than empirical: separating strategic LLM updates from low-level execution is a plausible control design worth testing more broadly. Multi-seed, longer-horizon, multi-model, and cross-domain experiments are the next steps needed to determine whether the pattern survives outside this testbed.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the authors used ChatGPT and Codex to assist with literature searching, manuscript restructuring, drafting and editing, presentation checks, and portions of code development and revision. The authors reviewed and edited the AI-assisted text and code, independently verified the scientific claims, numerical results, and cited references against the project artifacts and cited sources, and take full responsibility for the content of the article. The use of `gemini-3-flash` as an experimental decision-making component is part of the research method and is described separately in the experimental protocol.

References

- [1] E. Bonabeau, Agent-based modeling: Methods and techniques for simulating human systems, *Proceedings of the National Academy of Sciences* 99 (Suppl. 3) (2002) 7280–7287. <https://doi.org/10.1073/pnas.082080899>.
- [2] L. Tesfatsion, K. L. Judd (Eds.), *Handbook of Computational Economics, Vol. 2: Agent-Based Computational Economics*, Elsevier/North-Holland, 2006.
- [3] J. D. Farmer, D. Foley, The economy needs agent-based modelling, *Nature* 460 (2009) 685–686. <https://doi.org/10.1038/460685a>.
- [4] Y. Shoham, K. Leyton-Brown, *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*, Cambridge University Press, 2008.
- [5] S. Zheng, A. Trott, S. Srinivasa, D. C. Parkes, R. Socher, The AI Economist: Taxation policy design via two-level deep multiagent reinforcement learning, *Science Advances* 8 (18) (2022) eabk2607. <https://doi.org/10.1126/sciadv.abk2607>.
- [6] R. Buyya, C. S. Yeo, S. Venugopal, J. Broberg, I. Brandic, Cloud computing and emerging IT platforms: Vision, hype, and reality for delivering computing as the 5th utility, *Future Generation Computer Systems* 25 (6) (2009) 599–616. <https://doi.org/10.1016/j.future.2008.12.001>.
- [7] M. Armbrust et al., A view of cloud computing, *Communications of the ACM* 53 (4) (2010) 50–58. <https://doi.org/10.1145/1721654.1721672>.

- [8] O. Agmon Ben-Yehuda, M. Ben-Yehuda, A. Schuster, D. Tsafir, Deconstructing Amazon EC2 spot instance pricing, *ACM Transactions on Economics and Computation* 1 (3) (2013) Article 16. <https://doi.org/10.1145/2509413.2509416>.
- [9] R. S. Sutton, D. Precup, S. Singh, Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning, *Artificial Intelligence* 112 (1–2) (1999) 181–211. [https://doi.org/10.1016/S0004-3702\(99\)00052-1](https://doi.org/10.1016/S0004-3702(99)00052-1).
- [10] E. Altman, *Constrained Markov Decision Processes*, Chapman & Hall/CRC, 1999.
- [11] J. García, F. Fernández, A comprehensive survey on safe reinforcement learning, *Journal of Machine Learning Research* 16 (42) (2015) 1437–1480.
- [12] Y. Zhu, S. Qiao, Y. Ou, S. Deng, S. Lyu, Y. Shen, L. Liang, J. Gu, H. Chen, N. Zhang, KnowAgent: Knowledge-Augmented Planning for LLM-Based Agents, *Findings of the Association for Computational Linguistics: NAACL 2025* (2025) 3709–3732. <https://doi.org/10.18653/v1/2025.findings-naacl.205>.
- [13] M. Wen, Z. Wan, J. Wang, W. Zhang, Y. Wen, Reinforcing LLM Agents via Policy Optimization with Action Decomposition, *Advances in Neural Information Processing Systems* 37 (2024). <https://doi.org/10.52202/079017-3297>.
- [14] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, Y. Cao, ReAct: Synergizing reasoning and acting in language models, *International Conference on Learning Representations*, 2023.
- [15] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, S. Yao, Reflexion: Language Agents with Verbal Reinforcement Learning, *Advances in Neural Information Processing Systems* 36 (2023).
- [16] J. S. Park, J. O’Brien, C. J. Cai, M. R. Morris, P. Liang, M. S. Bernstein, Generative Agents: Interactive Simulacra of Human Behavior, *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology* (2023) 1–22. <https://doi.org/10.1145/3586183.3606763>.
- [17] X. Liu et al., AgentBench: Evaluating LLMs as agents, *International Conference on Learning Representations*, 2024.

- [18] X. Zhou et al., SOTOPIA: Interactive evaluation for social intelligence in language agents, *International Conference on Learning Representations*, 2024.
- [19] N. Li, C. Gao, M. Li, Y. Li, Q. Liao, EconAgent: Large Language Model-Empowered Agents for Simulating Macroeconomic Activities, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics* (2024) 15523–15536. <https://doi.org/10.18653/v1/2024.acl-long.829>.
- [20] L. Wang, C. Ma, X. Feng, et al., A survey on large language model based autonomous agents, *Frontiers of Computer Science* 18 (2024) 186345. <https://doi.org/10.1007/s11704-024-40231-1>.
- [21] T. Guo, X. Chen, Y. Wang, R. Chang, S. Pei, N. V. Chawla, O. Wiest, X. Zhang, Large Language Model Based Multi-agents: A Survey of Progress and Challenges, *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence* (2024) 8048–8057. <https://doi.org/10.24963/ijcai.2024/890>.